

# Triangle-triangle intersections with different numerical representations

Luca Garau \*  
University of Cagliari

Riccardo Scateni  
University of Cagliari

Gianmarco Cherchi  
University of Cagliari

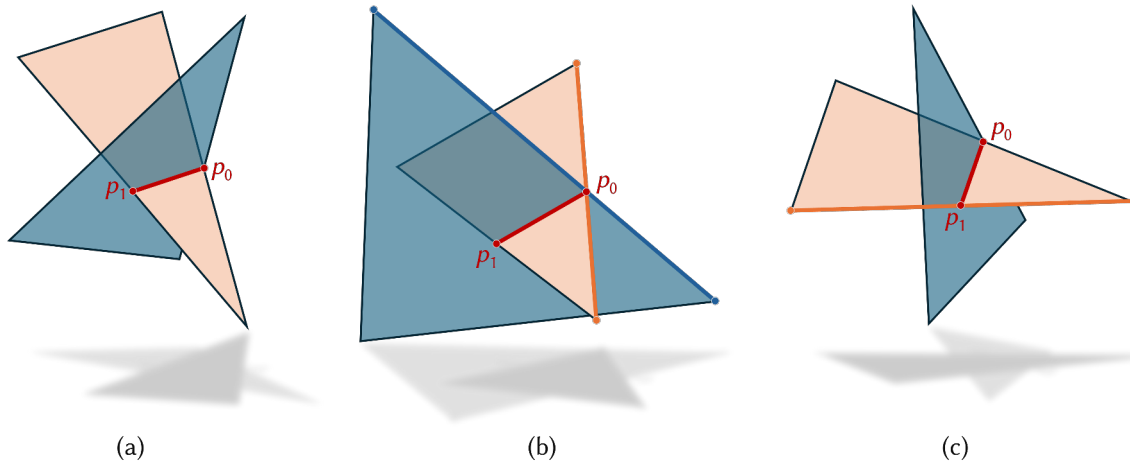


Figure 1: Intersections between triangles can be detected by checking intersections between simplexes of lower dimensions. In this figure, we can see: (a) a triangle-triangle intersection between a blue triangle  $T_0$  and an orange triangle  $T_1$ ; (b) the intersection point  $p_0$  is given by an intersection between an edge of  $T_0$  and an edge of  $T_1$  (segment-segment intersection); (c) the intersection point  $p_1$  is given by the intersection between an edge of  $T_1$  and the inner area of  $T_0$  (segment-triangle intersection).

## ABSTRACT

The detection and classification of triangle intersections in virtual environments are key ingredients in many computer graphics and geometry processing applications, especially for 3D reconstruction, rendering, modeling, and mesh optimization. Current solutions often necessitate rewriting intersection algorithms when transitioning between numerical representation systems, such as floating-point and rational numbers, to maintain robustness and precision. This work-in-progress report introduces a tool designed to address the triangle-triangle intersection problem robustly and exhaustively. By breaking down the problem into lower-dimensional simplex intersections and utilizing robust geometric predicates, our approach aims to provide a flexible, efficient, and general solution. The tool, developed using C++ templates, will support multiple numerical representation systems, ensuring adaptability without compromising accuracy. Upon completion, it will be benchmarked against state-of-the-art methods and released as an open-source resource, offering a trustworthy foundation for future advancements in research and development.

**Index Terms:** Mesh models, Collision detection, 3D reconstruction, Virtual reality

## 1 INTRODUCTION

Robust detection and classification of intersections between triangles in  $\mathbb{R}^3$  is a fundamental task in Computer Graphics and Geometry Processing, especially in 3D reconstruction, modeling or fixing of digital objects (meshes) for virtual environments.

\*e-mail: luca.garau@unica.it

Several problems can be related to the incorrect processing of triangle-triangle intersections in 3D models that touch or intersect each other. Examples can be rendering problems caused by mesh portions that share the same space (z-fighting) [15, 18], reconstruction of 3D scans, and in general, in many mesh processing/optimization/fixing algorithms [2].

Intersection detection and classification is a well-discussed problem in the state of the art [4, 8, 13, 17, 5], and poses complex challenges when robustness and correctness in the result of an algorithm are strict requirements. When using a simple floating-point representation, the correctness of the intersection detection and the computation of the new intersection point is not guaranteed due to the precision limitations that can lead to approximation errors. A typical case in which the problem arises is in the re-triangulation of intersecting (tris)surfaces, fundamental steps in algorithms like Mesh Arrangement [19, 13] of Mesh Booleans [6, 3]. This scenario is common in 3D reconstruction, where the calculated intersection points and segments are used to subdivide existing triangles and generate new mesh structures.

Errors in intersection analyses can lead to the generation of incorrect triangulations, causing geometric artifacts like degenerate or overlapping triangles and holes, that make resulting meshes unusable for further applications (see Fig. 3 in [6] for a real example). Hence, alternative numeric systems are often needed to represent triangle coordinates. In literature, we frequently find applications where the numerical representation of triangle coordinates is not done with the common floating-point numbers but with alternative representations like rational numbers [19, 16], or even implicit representations [1]. This means that every time we need to manage intersections between triangles with different coordinate representations, we have to re-implement and test all possible cases (which, contrary to what it may seem, are a very

high number).

Online, several pipelines for processing triangle intersections can be found, but they are always embedded in more complex tools, customized to solve specific problems with a specific predefined coordinate representation system. Reusing these codes for other purposes often requires considerable effort.

In this work-in-progress report, we describe how we are working to develop a robust and versatile solution for processing triangle intersections, able to identify and classify intersections accurately and independently from the numerical representation of the triangle coordinates. Our goal is to provide the community with a tool for detecting and classifying intersections that is exhaustive (without missing a single case), robust and templated with respect to the coordinate representation system we need to use. In this way, future researchers and developers who will have to tackle this problem will be able to count on a previously tested, reliable, robust tool, easily integrable into other algorithms and customizable with the desired type of numbers (choosing between floating-point or rational numbers, in the first version).

## 2 BACKGROUND AND RELATED WORKS

Finding intersections between 3D objects in a virtual environment in a robust way has always been a challenge. In the real world, we can always rely on infinite representable values, but when we represent objects in the computer world, we must deal with a discrete environment. The floating-point representation, the classic representation of numbers in computers, by construction transforms the space of representable numbers into a regular non-uniform grid. This means that even operations that we consider trivial can lead to an incorrect result, leading the flow of the algorithm in the wrong direction. Imagine, for example, the midpoint of a segment, which in the floating-point grid may not lie on the original segment, or the intersection between a segment and a triangle that may generate a point not inside the triangle or on the segment involved in the intersection. These are just a few examples of why we often need solutions other than the simple floating-point calculation to obtain a reliable result.

If we only want to determine whether there is an intersection between two elements, we can use geometric predicates with arithmetic filters [11, 14]. Such predicates can tell us exactly and quickly whether a point is to the left, right, or above a line (*orient2D*), or if a point is on one side or the other or lies on a plane (*orient3D*). Thanks to combinations of these predicates, we are able to robustly identify all types of intersections between triangles.

Once the intersections have been identified, it is necessary to calculate the new points generated by them to proceed, for example, with the computation of a new triangulation of the input triangles [10]. Here comes the real problem. If we calculate the intersection point using only floating-point numbers, we can run into approximation errors like the one described above, obtaining incorrect results (an example is depicted in Fig. 2). One way to overcome this problem is to use rational numbers [16] to represent the newly generated points. Rational numbers are represented as fractions composed of an arbitrarily long numerator and denominator that guarantee that “all” the numbers needed to achieve the correct result can be represented. On the one hand, this trick leads to correctness in the result, but on the other hand, it causes a significant slowdown in computation. In fact, representing points coordinates with rational numbers means that even elementary operations such as addition and subtraction involve operations between fractions with potentially enormous numerators and denominators. An alternative method for representing new points,

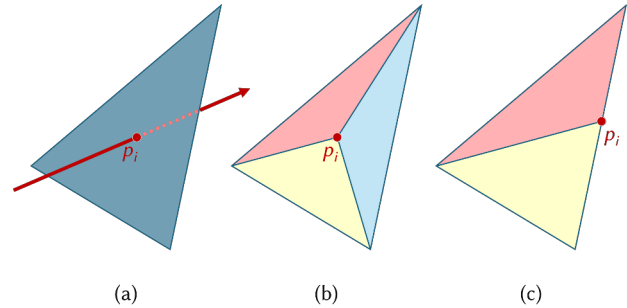


Figure 2: Approximation errors related to floating-point representation can lead to incorrect results even in the simplest cases. Suppose we want to subdivide a triangle using a point  $p_i$  given by the intersection with a segment that crosses it (a), the correct triangulation would be the one that divides the original triangle into 3 sub-triangles: yellow, blue, and red in (b). If the calculation of the intersection point is not precise (for example, it falls on an edge instead of within the area of the triangle), an incorrect triangulation is obtained, as shown in (c).

which we will not deal with in this project (at the moment), is an implicit representation of points [1]. If we are not interested in the exact coordinates of the intersection points but only in knowing which elements of the input it has been generated by, we can store the new point as a combination of lines and planes representing the segments and triangles of the input.

We can rely on several tools to overcome the approximation problem related to floating-point numbers. The problem remains that every time we decide to change the system of representation of triangle coordinates, we need to rewrite our pipeline to detect and classify the intersections. For this reason, we are developing a robust and fast algorithm, based on geometric predicates, that is able to process triangles whose coordinates can be represented both with floating-point notation or rational numbers and returns:

- All the information on the intersections found, including those between coplanar triangles (have you ever tried to enumerate how many there can be?)
- All the information required to create new intersection points (both with floating-point or with rational notation).

In short, it will no longer be necessary to rewrite the code from scratch based on the numeric representation to be used; it will be enough to call our code and set what type of coordinates it must process.

## 3 THE PROPOSED TOOL

This work-in-progress report presents the conceptual design of an algorithm for a versatile tool aimed at robustly detecting and classifying all possible types of triangle-triangle intersections, independently of the numerical representation used to represent the triangle coordinates and the coordinates of the points generated by the intersections.

An efficient approach to face this problem is to decompose the triangle-triangle intersection into intersections between lower-dimensional simplexes [12]. This modular strategy simplifies the problem by breaking it down into simpler and well-studied cases, such as coincident points, a point touching an edge, a point touching the inner area of a triangle, an edge crossing another edge, and an edge crossing the inner area of a triangle (see the

pseudo-code in Algorithm Algorithm 1). This subdivision in simpler simplexes allows us to (a) guarantee to find all the possible intersections between triangles (both no-coplanar and coplanar cases) and (b) optimize the code to make a robust and efficient pipeline. In Figure Fig. 1, we show an example of the detection of intersection points by simplifying the problem into simpler simplexes.

**Algorithm 1 “Triangle-triangle Intersection”.** This algorithm decomposes the triangle-triangle intersection problem into intersections between lower-dimensional simplexes. The algorithm takes as input the 6 vertices delimiting the two triangles  $T_0$  and  $T_1$  and returns a list  $L$  containing all the information about the detection intersections. In  $L$  we can have existing points or pointers to simplexes that crossing each other generate a new intersection point. Note that, in the pseudo-code notation, edges  $e$  are pairs of vertices of a triangle  $T$ .

**Require:** List of vertices  $V$  of two triangles  $T_0$  and  $T_1$

**Ensure:** List of information about detected intersections  $L$

```

1: for all vertices in  $V$  do
2:   check coincident vertices
3:   add coincident vertices in  $L$ 
4: end for
5:
6: for all vertices in  $V$  do
7:   if  $v_i$  of  $T_0$  touches  $T_1$  in edges or area then
8:     add  $v_i$  in  $L$ 
9:   end if
10:  if  $v_j$  of  $T_1$  touches  $T_0$  in edges or area then
11:    add  $v_j$  in  $L$ 
12:  end if
13: end for
14:
15: for all edges  $e_i$  of  $T_0$  and all edges  $e_j$  of  $T_1$  do
16:   if  $e_i$  crosses  $e_j$  then
17:     add info about intersection in  $L$ 
18:   end if
19: end for
20:
21: if  $T_0$  and  $T_1$  are coplanar then
22:   return list  $L$  ▷ no other type of intersections are possible
23: end if
24:
25: for all edges  $e_i$  of  $T_0$  and all edges  $e_j$  of  $T_1$  do
26:   if  $e_i$  crosses the area of  $T_1$  then
27:     add info about intersection in  $L$ 
28:   end if
29:   if  $e_j$  crosses the area of  $T_0$  then
30:     add info about intersection in  $L$ 
31:   end if
32: end for
33:
34: return list  $L$ 

```

To enhance the efficiency of the procedure, the algorithm will incorporate robust geometric predicates, such as Shewchuk’s 3D orientation predicates [14], to check the intersections. In the first version of the tool, our main goal is to comprehensively cover all possible cases of triangle intersections (do not miss any possible case of intersections), while in the second release, we will focus on optimizing the code to achieve efficiency, i.e., doing the minimum number of orient2D/3D calls to achieve the result.

A central feature of the proposed tool will be its independence from the numerical representation. We are developing our tool

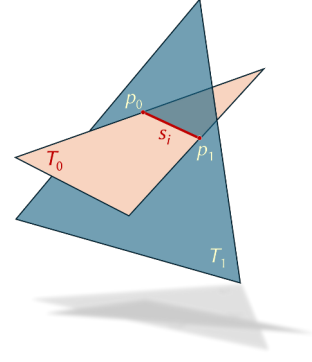


Figure 3: Intersections between triangles does not necessarily generate only new points. In some cases, intersection segments can also be created. In this example, triangle  $T_0$  intersects triangle  $T_1$ , generating the two intersection points  $p_0$  and  $p_1$ , which are also the endpoints of the new intersection segment  $s_i$ .

by designing all the functions to detect and classify intersections using C++ templates. These templates are expected to support different numerical systems, such as floating-point numbers (for performance-critical applications that tolerate some imprecision) or rational numbers (for applications requiring exact precision, as supported by libraries like CGAL [16]). Templates will allow the flexibility to adapt the algorithm to operate with any numeric type, from basic types like float and double to custom rational types, without requiring substantial changes to the core logic. Indeed, we are preparing the code so that, in a second release of the tool, it will be possible to perform calculations using implicit representations of triangles and intersection points [1].

Our algorithm will be structured as a unique function for the final user, taking the information about two triangles as input and returning the information about the point and segments generated by the intersection (if any) as output. Indeed, not only points but also segments can be the results of two intersecting triangles, as depicted in the example in 3.

The output of the intersection detection function will be as generic as possible, meaning it will not depend on any specific numeric representation, thereby facilitating integration with diverse applications. The tool will return an `IntersectionResult` object containing:

- *Intersection Points*: the list of sub-simplexes that, by intersecting, generated an intersection point;
- *Intersection Segments*: the pair of intersection points that, when joined, form a new segment;
- *Additional Metadata*: information such as coplanarity of the triangles and the total number of intersection elements found.

The output information can then be used to represent the new points and segments with the desired numeric representation as explicit (e.g., floating-point or rational) or even implicit forms (e.g., implicit points of [1]).

Although the tool is still under development, the outlined features aim to provide a robust and reusable solution to the problem of triangle-triangle intersection detection and classification. The implementation will be followed by an extensive test phase with famous geometric tools [7], and with the incorporation of our code

into famous state-of-the-art algorithms for 3D reconstruction and virtual reality applications, in order to guarantee the robustness of the developed solution and its easy integration into existing or future algorithms. We are developing our tool in C++, using the Cino-lib [9] library for the basic geometric utilities, and we plan to redistribute the code in an easy-to-integrate and header-only open-source solution.

#### 4 CONCLUSION AND FUTURE WORK

In this article, we have presented our ongoing project aimed at developing a robust and exhaustive pipeline for detecting and classifying intersections between triangles in a virtual environment. Our proposed algorithm is designed to be independent from the numerical representation used for triangle coordinates, ensuring easy integration into new tools and availability to the community as open-source software.

The first release of our code will be rigorously tested against state-of-the-art algorithms. In subsequent releases, we plan to enhance the code by adding support for implicit point representations. Furthermore, we will focus on optimizing the code to achieve not only robustness but also optimal efficiency.

#### ACKNOWLEDGMENTS

The authors gratefully acknowledge the support to their research by project “FIATLUCS - Favorire l’Inclusione e l’Accessibilità per i Trasferimenti Locali Urbani a Cagliari e Sobborghi” funded by the PNRR RAISE Liguria, Spoke 01, CUP: F23C24000240006.

#### REFERENCES

- [1] M. Attene. Indirect predicates for geometric constructions. *Computer-Aided Design*, 126:102856, 2020. doi: 10.1016/j.cad.2020.102856 1, 2, 3
- [2] M. Attene, M. Campen, and L. Kobbelt. Polygon mesh repairing: An application perspective. *ACM Comput. Surv.*, 45(2), Mar. 2013. doi: 10.1145/2431211.2431214 1
- [3] G. Bernstein and D. Fussell. Fast, exact, linear booleans. *Computer Graphics Forum*, 28(5):1269–1278, 2009. doi: 10.1111/j.1467-8659.2009.01504.x 1
- [4] J.-W. Chang and M.-S. Kim. Efficient triangle–triangle intersection test for obb-based collision detection. *Computers & Graphics*, 33(3):235–240, 2009. 1
- [5] G. Cherchi, M. Livesu, R. Scateni, and M. Attene. Fast and robust mesh arrangements using floating-point arithmetic. *ACM Transactions on Graphics (TOG)*, 39(6):1–16, 2020. 1
- [6] G. Cherchi, F. Pellacini, M. Attene, and M. Livesu. Interactive and robust mesh booleans. *ACM Transactions on Graphics*, 41(6), 2022. doi: 10.1145/3550454.3555460 1
- [7] D. Eberly. Geometric tools. <https://www.geometrickit.com>, December 2024. 3
- [8] P. Guigue and O. Devillers. Fast and robust triangle-triangle overlap test using orientation predicates. *Journal of graphics tools*, 8(1):25–32, 2003. 1
- [9] M. Livesu. cinolib: a generic programming header only c++ library for processing polygonal and polyhedral meshes. *Transactions on Computational Science XXXIV*, 2019. <https://github.com/mlivesu/cinolib/>. doi: 10.1007/978-3-662-59958-7\_4 4
- [10] M. Livesu, G. Cherchi, R. Scateni, and M. Attene. Deterministic Linear Time Constrained Triangulation using Simplified Earcut. *IEEE Transactions on Visualization and Computer Graphics*, 28(12):5172–5177, 2021. doi: 10.1109/TVCG.2021.3070046 2
- [11] B. Lévy. Robustness and efficiency of geometric programs: The predicate construction kit (pck). *Computer-Aided Design*, 72:3–12, 2016. 23rd International Meshing Roundtable Special Issue: Advances in Mesh Generation. doi: 10.1016/j.cad.2015.10.004 2
- [12] B. Lévy. Exact predicates, exact constructions and combinatorics for mesh csg, 2024. 2
- [13] T. Möller. A fast triangle-triangle intersection test. *Journal of graphics tools*, 2(2):25–30, 1997. 1
- [14] J. Richard Shewchuk. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 18:305–363, 1997. 2, 3
- [15] P. Setthawong. Potential z-fighting conflict detection system in 3d level design tools. In *Information Science and Applications*, pp. 279–285. Springer, 2015. 1
- [16] The CGAL Project. *CGAL User and Reference Manual*. CGAL Editorial Board, 6.0.1 ed., 2024. 1, 2, 3
- [17] O. Tropp, A. Tal, and I. Shimshoni. A fast triangle to triangle intersection test for collision detection. *Computer Animation and Virtual Worlds*, 17(5):527–535, 2006. 1
- [18] A.-A. Vasilakis and I. Fudos. Depth-fighting aware methods for multi-fragment rendering. *IEEE transactions on visualization and computer graphics*, 19(6):967–977, 2012. 1
- [19] Q. Zhou, E. Grinspun, D. Zorin, and A. Jacobson. Mesh arrangements for solid geometry. *ACM Trans. Graph.*, 35(4), July 2016. doi: 10.1145/2897824.2925901 1